

Admissible: A Fail-Closed Admissibility Kernel for Agents That Do Not Repeat

Roque Briceño

Version 0.8.1. 4 September 2026. Licensed under CC BY 4.0.

Companion reports: *Fail-Closed Class Dispatch* (layer I) and *Refutation-Gated Admission* (layers R and C). Each report stands on its own, with its own invariants, proofs and review rounds; this paper is the composition — one machine, one predicate, and the two theorems the composition earns. It adds framing and two composition results, not new mathematics, and says so.

Abstract

A system that delegates work to large language models cannot certify an output by naming its author: the same worker, prompt and context produce a different artifact on every run. We present an integrity kernel for this setting, built as three fail-closed transition systems composed by non-interference, whose union of guarantees is the soundness of a single query, *admissible(id)*, and the loudness of its complement.

Layer **I** (fail-closed class dispatch) proves *identity*: a stage passes only when the executed model equals the one bound from a class-scoped allow set, with no fallback edge; seventeen invariants over the binding, envelope, receipts and memory promotion. Layer **R** (refutation-gated admission) proves *scrutiny*: an artifact seals only when declared, deterministic, replayable refuters — pinned before generation, measured against named defect models, run over k concordant samples — were given a fair chance to kill its claims and failed, with the measured power carried on the seal and never raised afterwards. Layer **C** (the escape ledger) proves *standing*: a defect later found in sealed work, demonstrated as a counterfactual trial of the very instrument the seal trusted, mechanically impeaches the seal, charges every vouching refuter, and can never be silently forgotten by any successor policy.

The composed kernel is a reference monitor for machine work-products, in the classical sense: complete mediation of the stores, journals whose replay re-derives every transition and compares it to the supplied record field by field — refusing inconsistent alteration, forgery and duplication, while coherent root rewrites and deletion require a previously trusted authenticated head — and verifiability by construction — every guard is a named method that a mutation suite deletes one at a time, proving each individually load-bearing, and every proof cites the file and line that enforces it, bound by a test. The guarantees are about the *record*, never about truth: what is proved is that the paperwork cannot lie about who ran, what was survived at what measured strength, and what has been found against it since. This is the admissibility standard of evidence law — provenance, known error rate, controlling procedure, impeachment — enforced by a kernel, and it deliberately certifies exactly what a court's admission does: procedure, never truth.

1. Introduction

1.1 The problem

Delegation to stochastic generators breaks the oldest assumption in software integrity: that knowing *which* component ran tells you *what* it did. A deterministic function's identity determines its output; an agent's does not. Three questions follow, in order, and each is the whole subject of one layer:

1. **Who actually did the work?** Control planes name one model while data planes call another; a dead bind can silently continue on a fallback; a reviewer can be the author. Without identity, nothing downstream is attributable.

2. **What was the work subjected to before acceptance?** A green check from an empty test suite and one from a suite that kills 95% of seeded defects print identically. Survival without measured severity launders weak scrutiny into strong.

3. **What happens when accepted work turns out to be wrong anyway?** In most pipelines: nothing mechanical. The miss is fixed, the instruments that vouched keep vouching, and the next test battery is free to forget the defect class entirely.

1.2 The predicate

The kernel's guarantees compose into one query, exposed by the implementation and defined here:

admissible(id) := id ∈ S_R ∧ mediated(id) ∧ ¬ tainted(id) ∧ ¬ impeached(id)

where S_R is the sealed store (written only by Seal, which requires the identity layer's Accept), *tainted* means a refuter the seal relied on was later caught nondeterministic, and *impeached* means a valid escape stands against a sealed claim. Each conjunct is the top of one theorem family: membership in S_R rests on identity (I1–I17, inherited by citation) and scrutiny (R1–R13); the two negative conjuncts rest on standing (C1–C7). The project takes its name from this predicate.

The name is also an argument. Evidence law admits testimony not because it is true but because it satisfies *procedure*: established provenance and custody, a known error rate, controlling standards, and exposure to cross-examination — with impeachment available afterwards. The Daubert factors for expert evidence (testability, known or potential error rate, maintenance of standards; *Daubert v. Merrell Dow*, 1993, itself citing Popper) map onto this kernel exactly: refuters are the testability requirement, carried power is the known error rate — in ledger mode exact on the named defect model D, in bounded mode a computed function of declared (ϵ , N) and no more trustworthy than ϵ ; either way a claim about the instrument, never the generator's real-world error rate, pinned versioned policy is the controlling standard, and the escape ledger is impeachment. The analogy is honest precisely where the theorems are: admissibility certifies procedure, never truth, and the seal says so on its face.

1.3 Contributions

1. A three-layer, fail-closed transition system in which **identity, scrutiny and standing** are each enforced by a kernel with inductive invariants (I1–I17, R1–R13, C1–C7), composed by two non-interference lemmas (R11, C7) so that each layer's proofs survive composition unchanged (§6).

2. **Measured doubt as a carried record**: power enters a seal only as a kernel-counted kill rate against a content-addressed defect model, or a computed bound from declared, seal-visible parameters — never as a scalar anyone wrote down — write-once, with composition across refuters only by measured union or labelled max (§4). The two sorts are never collapsed into one another, and the power floor is compared against the *weaker* sort present, so a declared figure cannot clear a floor that a measurement failed.

3. **Escapes as counterfactual trials**: a post-acceptance defect report that is a re-run of the *pinned* refuter at a finder-chosen input, adding no trust assumption the seal did not already carry; from it, non-discretionary impeachment, per-cell charge totality, a coverage ratchet on policy succession, and query-shaped demotion (§5). To our knowledge the non-discretion and charge-totality properties have

no prior formalization (§10).

4. A **verification methodology** matched to the claims: every guard is an individually deletable method with a red-first deletion proof; every proof citation is a file:line:symbol triple bound by a move-detecting test; premises were attacked by adversarial review *before* each kernel was written and the finished kernels re-attacked after, with all confirmed findings applied and journaled (§7).

5. An honest boundary, stated as theorems' complement: an explicit, consolidated *what is not proved* (§11), including the two empirical hypotheses the numbers rest on (defect-model coupling; corpus representativeness) and every trust assumption (A0–A13, B0–B14).

2. Setting and threat model

Parties. An *operator* owns policy and adjudication (trusted for governance, made visible, not defeated — B13). *Specialists* (generators) are untrusted and stochastic. *Refuters* are deterministic programs under a pinned hermetic runtime (B2). A *harness* executes and reports (honest per B1/A10 — the classical adapter boundary; its lies about deterministic components are falsifiable by replay, which is strictly more than can be said of its lies about stochastic ones). *Finders* of escapes are untrusted; their filings are checkable.

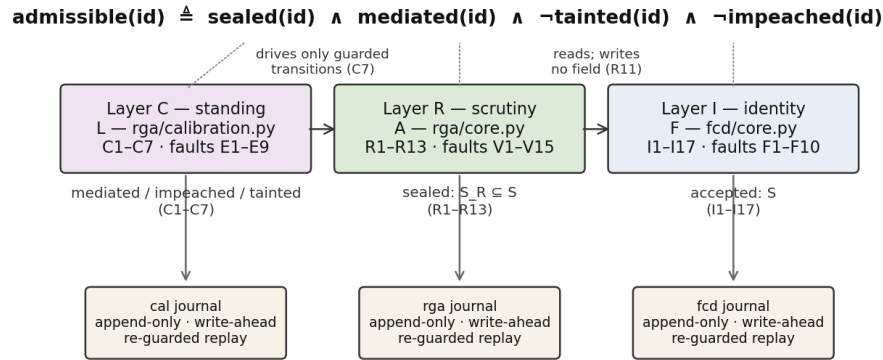
Machines. Three state machines with disjoint write sets, composed left-to-right by read-only observation and guarded interposition:

F (fcd/core.py) ← A (rga/core.py) ← L (rga/calibration.py)

A reads F's state and writes none of it (R11); L reads both and writes neither directly, driving A only through its own guarded transitions (C7). Every transition appends to an append-only journal; every machine rebuilds deterministically from its journal, re-checking on rebuild what it checked live (a property earned the hard way: the first two replay implementations were broken by review and repaired — §7).

Adversary. In scope: any sequence of public transitions by untrusted parties — mis-binding, fallback hopping, claim-shaping, refuter-shaping, tailoring outputs to known tests, witness multiplication, corpus flooding with true filings, inconsistent journal tampering detected at replay, and selection across retries. Coherent history replacement or deletion is detected only relative to a previously trusted authenticated publication. Out of scope, stated as assumptions: physical execution fidelity (A10/B1), runtime hermeticity (B2), hash second-preimage (A11/B7), author-identity strings (B11), operator governance (B13), authenticated-publication key/registry/bootstrap integrity (B14), and post-seal byte availability (B12). The full assumption tables are ../INVARIANTS.md §0 and ../RGA/INVARIANTS.md §0/§8.0.

Figure 6. One machine, three layers, one predicate



Each machine rebuilds from its journal re-checking every live guard; inconsistent records are refused, while historical authenticity requires an external current head.

3. Layer I: identity (fail-closed class dispatch)

A work item has a class c and a frozen body; policy gives $\pi(c)$, $\delta(c)$ and $\phi: A \rightarrow M$. The machine's rows are Open, Admit, Bind, Observe, Pass/PassRefuse, Close, Retry, Accept — with the load-bearing split that Bind writes only the *declared* identity and Observe alone writes the *executed* one, so the Pass guard $\text{norm}(m_{\text{exec}}) = \text{norm}(m_{\text{decl}})$ compares two independently sourced witnesses and can genuinely fail.

Selected theorems (full statements and proofs: ../INVARIANTS.md, ../PROOFS.md):

- **I1** $\text{pc} = \text{Passed} \Rightarrow \text{norm}(m_{\text{exec}}) = \text{norm}(m_{\text{decl}}) = \text{norm}(\phi(a))$.
- **I3** No Passed state has $\text{norm}(m_{\text{exec}})$ outside $\{\text{norm}(\phi(x)) : x \in \pi^*(c) \setminus \delta(c)\}$ — this machine has no leftover-hop edge.
- **I5/I8** Accepted means every required stage Passed; the store has exactly one writer.
- **I6** A check-stage admit excludes the item's authors: dual control.
- **I10–I17** pin the context envelope: package hashes, receipts, steering acknowledgement and cache identity all bind to the current attempt and nonce; memory promotes only through serialized compare-and-swap by accepted work.

Layer I is Clark–Wilson's enforcement rules applied to LLM binds — CDIs, TPs, validated writes, separation of duty — and its paper says so rather than claiming a new access-control algebra. Two of its guards were repaired during this project's reviews (a pc guard on NoAdmit; a fresh-id guard on Open), because the upper layers' proofs lean on I4 and I8 as *state* invariants; the repairs carry red-first tests.

4. Layer R: scrutiny (refutation-gated admission)

Layer I verifies the die, not the roll. Layer R moves the gate to the claim. A class pins, at Open and before any generation: formal claims (spec-hashed; the generator never authors what it is measured against), the refuter versions that attack each claim, sample count k , concordance threshold θ , and

power floor $p_{\min}$.

Samples are k FCD-passed write stages of one item under one bind key — identity inherited per sample by citation of I1/I2/I3 — each registered (kernel-hashed, single-holder) before the next stage may run, with a post-artifact nonce. **Trials** are one refuter against one claim on one sample, seeded by $H(v_i | h_i | r | \text{claim})$ so no seed exists before its artifact; verdicts are refuted/survived/inconclusive and only survival counts. **Power** enters only as a record: kills counted by the kernel over a ledger against a content-addressed defect model D whose id-set *and* *author* are fixed by first record, exact on D ; or $1-(1-\epsilon)^N$ computed from declared (ϵ, N) that the seal carries. **Concordance** is agreement of witness vectors with the *designated* sample (stage 0, fixed before anything ran) — a precondition that closes the gate, never an election.

Selected theorems (full: ../RGA/INVARIANTS.md §3, proofs with line citations in ../RGA/PROOFS.md):

- **R1** No admission without survival in every (claim, refuter, sample) cell; refuted and inconclusive close, published.
- **R2** Power is carried, never inferred: every figure on a seal equals a write-once, kernel-computed record; nothing raises it after the fact; composition across refuters is measured union over a shared D or a labelled max — never a product.
- **R3** Separation of duty, extended: refuter fixed before generation; authored by neither generator nor defect-model author; generator's package provably excluded refuter material; samples interleaved with their stages so slots cannot be assigned after seeing the batch.
- **R4** Concordance against the designated sample at θ , with (agreeing, k) on the seal so $k \geq 1$ is visibly trivial.
- **R5/R6** A replay divergence refuses a refuter monotonically, everywhere; every refuter used was replayed at least once on this line.
- **R8** $S_R \subseteq S$: everything sealed was first accepted by layer I .
- **R11** A writes no field of F — the composition lemma.

The seal carries: the artifact hash; per claim, each refuter's mode, power, D (or ϵ, N), kills and $|D|$; the composite and its label; (agreeing, k); the sampling configuration; the FCD identity; and an explicit list of what was *not* attacked, with its disposition. Faults V1–V15 name every forbidden step; V1–V5 publish, the rest refuse.

Figure 7. Anatomy of a seal (write-once; nothing on it can be raised later)

identity (layer I, by citation)	work item · class · frozen body hash · fcd policy version · generator · executed model
artifact	hash of the exact bytes accepted — the served bytes, byte-checked at Accept
sampling	k samples · designated sample = stage 0 · sampling config hash · post- artifact nonces
per claim	claim id · spec hash (authored outside the generator) · pinned refuter versions
per refuter	mode (ledger bounded) · power = kills/ D against content-addressed D (author fixed by first record), or $1 - (1 - \epsilon)^N$ from declared (ϵ , N) · kills · D
composite	union over shared D, or labelled max — never a product · (agreeing, k) so k=1 is visibly trivial
residual	every claim NOT attacked, with disposition: check_stage unreviewed — the seal names what it does not cover
standing (layer C, queries)	mediated? impeached? tainted? — never stored on the seal; computed against the retained escape ledger, with authenticated current heads making deletion of an anchored prefix loud

5. Layer C: standing (the escape ledger)

R leaves one loop open, and its own limits name it: kill-rate on D is power against real defects only under the coupling hypothesis, and nothing forces a found miss to have consequences. Layer C closes the loop that can be closed.

An **escape** against a sealed claim is, in its automatic form (tier A), a *counterfactual trial*: the claim's pinned refuter re-run at a finder-chosen nonce on the sealed bytes — the finder supplies only the nonce and the bytes; the kernel hashes the bytes against the seal, derives the seed itself, and requires verdict refuted plus one identical replay. Kills are sound regardless of proposer, and this channel adds no assumption the seal did not already carry. Any other checker is tier B: no effect of any kind until a named, journaled adjudication — the per-escape claim-fidelity judgment made visible rather than pretended away.

From one valid escape, mechanically:

- **C3 Impeachment entailed, un-revocation attributed**: impeached(id) is entailed by the escape — a pure query; the seal is immutable; no authority can decline the revocation, because the revocation carries its own replayable proof. The upward direction is deliberately harder and is the theorem's real content: an established escape's effect is removed only by a divergence on *that* run, which merely marks it contested and leaves it impeaching, plus a named, journaled resolution deciding *void*. No single report — about another run, or from the scrutiny layer's refusal registry — raises a line's

standing. A replay divergence is a report about a component the stack only assumes deterministic (B1, B2), so when two reports about one run conflict the kernel has no ground to prefer either; what it can do, and now does, is refuse to let the later one win in silence. §7 records that the first version of this layer did let it, and what that cost.

- **C2 Charge totality:** every refuter that vouched is charged — read from the journal, never declared — exactly once per (line, claim, refuter\ version) cell, so witness and checker multiplicity cannot inflate.

- **C4 The ratchet:** no successor policy installs unless its ledger defect models cover the class's valid corpus minus journaled, attributed exclusions; a class owing coverage cannot be dropped; the install event (which now exists) carries the primaries and the diff. *Forgetting is loud.*

- **C6 Demotion:** charges past a declared budget block new pins unconditionally and gate sealing where the class declared it — a pure function of the valid charge set, falling only where C3's upward direction allows, never an event cascade, never a rewrite. A checker's discredit or refusal no longer lowers the count: a suspect instrument stays charged rather than being cleared by a report about itself.

- **C5** Every seal issued through the authority is stamped, same-step, with its instruments' track record and the corpus's finder-provenance split — primaries, never a rate, with the stated reading that zero charges is absence of filed evidence.

- **C7** L writes no lower-layer field directly — the second composition lemma.

6. The composed guarantees

The unified results are compositions, proved by citation of the layer theorems plus the two non-interference lemmas. They are the paper's claim stated once:

Theorem 1 (Soundness of the record). If $\text{admissible}(\text{id})$ holds at journal position t , then the journals up to t contain, re-derivably: (i) for each of the k samples, a bind and an observation whose normalized identities agree with ϕ of a specialist admitted from $\pi^{\wedge}(c) \setminus \delta(c)$ [I1–I3, via R7]; (ii) *for every pinned claim and refuter, a surviving trial on every sample and at least one replayed trial per refuter on this line, seeded after the artifact it attacked, under power records the kernel computed, with concordance $\geq \theta$ against the designated sample and composite power $\geq p_{\min}$ [R1–R10, R12–R13]; and (iii) no valid escape standing against any sealed claim, no reliance on a refuter caught nondeterministic, and a same-step stamp of every instrument's charge record as of sealing — required by the predicate itself, not merely emitted beside it: a seal the calibration authority never mediated carries no stamp, and admissible refuses it as layer IR rather than IRC [C1–C3, C5].* Proof.* Each conjunct of admissible is the conclusion of the cited family; R11 and C7 guarantee the families hold on the combined trace because each upper machine's writes are disjoint from the lower machines' state. QED

Theorem 2 (Loudness of deviation). Every transition sequence that would falsify a conjunct of Theorem 1 either writes nothing (a guard refusal: faults F5–F10, V6–V15, E4/E6/E7/E9 raise) or appends a named fault to the journal before any effect (F1, F3, F4, V1–V5, E5 publish; F2 has no runtime-instance field to fire on and is listed as unproved, and E1–E3 and E8 are query-time validity and writer inspection rather than transitions, so they appear in neither bucket); and replay is *transition-local*: each of the three from `_events` re-checks, before every write, the guards live execution ran, so derived/output fields altered inconsistently, forged or duplicated transitions, reordering against preconditions, and removals that later events recompute against are refused — and every journal a live machine accepted replays unchanged. Replay does not authenticate which internally valid history is the anchored one. **Unauthenticated-history residue** includes a coherent rewrite of root transition inputs, a journal shortened at the tail, or a coherent removed group no surviving event recomputes against; each

is indistinguishable from another honest history to replay alone. Deletion is the direction that can *raise* standing, since dropping an escape un-impeaches its line and dropping a refusal un-taints every seal that relied on it — and since the un-revocation repair of §7, a refusal group is in that class unconditionally, because no standing reader recomputes against a refusal any more. Replay alone cannot close it. v0.5 adds a conditional authenticated-publication theorem: immutable authority-owned journals are sealed into three namespace-scoped ordered heads, successor receipts prove extension of the anchored cumulative event chain, the heads are accepted atomically, and the exact heads plus kernel-derived I/R/C predicates are bound into one signed receipt; verification rejects stale, coherently shortened, or longer-forked records. The theorem applies only when callers issue and verify that receipt, and still trusts signer/key custody, registry monotonicity, and the single-writer authority model. One appended event has the same character: a calibration stamp forged for the most recent unmediated seal contradicts nothing earlier, so it is accepted, while one forged for any earlier seal breaks stamp order and is refused. Finally, membership checks on rebuild read the final Admission registry, so an install refused live for a missing Measure record can replay once a later record supplies it. Everything else in the transition-consistency class — inconsistent derived fields, forged transitions, duplicates, and deletions any later event recomputes against — is refused, because each machine compares its re-driven transitions to the journal field by field rather than trusting the file. *Proof.* By the fault tables and their executable checks: every V- and E-fault names its enforcing method, and tests/ demonstrates, for each method singly, that the forbidden state is unreachable with it and reached without it; every F-fault is driven to its refusal by the identity kernel's transition tests (tests/test_invariants.py, tests/test_core.py), which exercise the same table without per-guard deletion. QED

Neither theorem mentions truth. That is the point, and the limit (§11).

7. Methodology: executable proofs and adversarial rounds

The verification discipline is a contribution independent of the theorems it checks:

1. **Delete-the-guard.** Every guard of the scrutiny and standing kernels is a named method. For each, the suite runs a scenario twice: with the guard intact (forbidden state unreachable) and with the guard replaced by a no-op (forbidden state reached). A guard no deletion turns red fails the suite as dead code — and the discipline is applied to itself: clauses proved unreachable by construction were removed and derived instead, with the removals documented. The identity kernel predates this discipline: its guards are driven to refusal by transition tests over the same fault table, without per-guard deletion — a difference stated, not papered over.
2. **Citation binding.** Every R- and C-proof step cites file:line:symbol; a test opens the file and fails if the line no longer contains the symbol (the layer-I proofs predate the binder and carry file-level citations only). A move-detector, not a semantic check — the sentence claims enforcement, the test keeps it pointed at live code. It fired repeatedly during this project's own refactors, exactly as designed.
3. **Premise before kernel.** Each layer's design brief was attacked by independent adversarial reviewers *before* any code — five lenses and 38 verified findings for R; four lenses and 28 for C — with every finding handed to a skeptic instructed to refute it. Two of the author's pre-registered designs were killed at this stage (a Wilson-interval power bound; sibling-item sampling), which is the cheap time to kill them.
4. **Review after build.** The finished kernels were re-attacked (five lenses for R; two for C). Both rounds returned REVISE; all confirmed findings were applied, including five reproduced R-kernel defects, three of which invalidated published proofs, two latent soundness bugs in layer I itself, and a C-replay that trusted its own journal. Every round is journaled in eval/reviews/, and every repair carries a red-first

test.

5. Attack the finished kernel with its own theory. The discipline above was then turned on the released kernel by computing the *polarity* of its event alphabet — for the standing predicate, which event types can only raise it, which can only lower it, and which are neutral. An event that raises a standing predicate is a candidate soundness defect, because the design intends exactly one such event class. The computation found four, and they are reported here rather than only in a changelog, because a methodology's value is what it catches:

- **A single party could un-impeach everything a checker had impeached.** A divergent replay discredited the checker, and validity voided every run of that checker, established or not. One party holding the sealed bytes could file a junk tier-A escape, self-replay it divergently, and lift every impeachment that checker had established — with nothing tainted and an empty refusal set. This contradicted C3 as published, and the transition table had always said only *unestablished* escapes were void. A second channel had the same shape through the refusal registry. The repair is C3's stated asymmetry, and it removed the clause rather than patching it: an *unestablished* run already fails the establishment conjunct, so voiding *unestablished* runs of a discredited checker is unreachable, and voiding established ones is what the table forbade.

- **A declared figure could clear a floor a measurement failed** (§4): bound accepts $(\epsilon, N) = (1, 1)$, whose figure is 1.0, and the cross-sort max let it satisfy any $p_{\min}$ over a $O(|D|)$ ledger. Banning $\epsilon=1$ would have been a symptom fix, since $(0.9999, 10)$ reaches the same figure; the sorts are now never collapsed.

- **Two halves of one replay seam:** rebuild re-checked one filing guard against the *final* registry, so an honest journal whose checker was refused later was refused on rebuild; and it omitted the refusal check the live guard makes, so a filing by an already-refused checker was refused live and accepted on rebuild. A filing now records the scrutiny cut it was written at, bounded at both ends and forbidden to move backwards.

One repair has a cost, and it belongs here rather than in a footnote. While a refusal voided the refused checker's established escapes, later stamps and policy installs recomputed against it and therefore *anchored* it. Now that validity never reads the refusal registry, no standing reader depends on a refusal, so refusal groups are freely deletable and the unauthenticated-history residue of §6 is one event class wider than before. That is preferable to fail-open revocation, and closing it takes the authenticated publication path rather than replay.

Current state: 45 per-guard deletion proofs and 2 joint cuts covering the scrutiny and standing kernels, plus 2 citation binders over the R- and C-proofs; the identity, scrutiny, standing, custody and composition suites are green. Counts are re-measured from the two mutation registries rather than remembered, because a stale count is exactly the kind of claim this discipline exists to catch.

8. Implementation

Three kernels, pure Python 3.10+, no I/O, injectable clocks and nonce sources: `fcd/core.py` (identity), `rga/core.py` (scrutiny), `rga/calibration.py` (standing). State enters only as arguments; every fact about the world is a report crossing a named assumption; every transition appends journal events matched by an explicit contract (`metrics/SCHEMA.md`); every machine rebuilds from its journal with re-guarded replay. The reference server and cockpit sit above the kernels and cannot pass, accept, seal or promote; a skin is a read-only projection. The store queries a deployment must consult — admissible, mediated, `check_dependencies`, `suspect`, `tainted`, `impeached` — are pure functions of journal state.

9. Evaluation protocol

Rates are computed only on a *named cut* $[t_0, t_1]$ with a class-derived window, from a write-ahead journal, with policy evaluated as-of each event — otherwise zeros are estimates biased clean, not results. Under that rule the repository carries three empirical studies, of decreasing internal validity and increasing external relevance, and the full account of every run — including the ones that turned out to be measuring the instrument — is `eval/LOG.md`, an append-only log whose convention, from the first real-defect detection run onward, is that each run's hypothesis, metric, adjudication rule and stopping rule are committed *before* the run and its result appended after; the four earlier entries are marked retrospective in their own headers.

The kernel bench (`eval/bench/`, Figure 8) drives all three layers over reference implementations of four stdlib-specified functions (median, wrap, normpath, quantiles), with the standard library as the strong refuter's differential oracle and a seeded AST mutator building the defect models. Its numbers — carried power, seal/refutation/discord outcomes, escapes established, demotions, ratchet coverage — are kernel queries on the named cut, reproduced byte-identically by `tests/test_bench.py`. Two of its three generator conditions are close to definitional and the figure now says so: the *honest* sample **is** the reference implementation that serves as the oracle, so 32/32 sealing is true by construction, and an honest line that failed to seal would be an instrument fault — as one was, when a property refuter flagged `normpath("/")` and refuted the reference implementation itself. The *sloppy* mutant is drawn from the defect model D that power is measured against, so its refutation is a within-model result that says nothing about a defect outside D. What the bench does establish is that the machinery is internally consistent under adversarial conditions, and it establishes one thing that is not definitional: **every latent-defect seal was impeached afterwards, 7 of 7**, while three unstable seals were not and stand as misses. That is layer C working, and its residue, on the same figure.

A real generator (`eval/generators/`) replaced the simulated one for 48 samples across those four targets, recorded into content-addressed corpora and replayed through the same seam, so determinism survives. **Not one sample was defective**, across 12 textually distinct `normpath` implementations, all 48 matching their oracle on 3,000 generated inputs. The gate refused none of them: 16 of 16 lines sealed, zero false refusals, which is the first measurement of the false-positive half against code a generator actually wrote — read with the corpus's own limitation, that nothing committed establishes the k samples within a line were independent draws, so concordance there is untested and the discord layer is not exercised by this corpus. The detection half could not be measured, and the reason is structural rather than a matter of adding targets: a target whose oracle is a well-known library function is one whose answer a modern generator recalls rather than derives, so the differential refuter ends up comparing the oracle to itself. No claim about any model follows; the corpora name no vendor and the run used one configuration throughout.

Real historical defects (`eval/realdefects/`, Figure 9) are the study that bears on coupling directly: 63 module-level functions drawn from `BugsInPy`, of which 42 could be run as a differential between their buggy and fixed commits under one interpreter, with inputs found by search rather than sampling and with no knowledge of the bug. It yields **eight defects separated and then hand-verified against the patch that closed them**, in `youtube-dl`, `scrapy` and `thefuck`, each with a re-runnable witness. It does **not** yield a detection rate, and the figure is drawn to refuse one. Five successive runs each carried an instrument fault invisible in its own summary statistic, in this order: a stubbed parent package that cost two thirds of the corpus, a filter too narrow, an argument mutated between the two sides, a filter too broad, and finally an artifact rule that measurement afterwards showed never fires at all. Of the last run's 42 testable candidates, three are excluded by a guard bug rather than by the corpus, two of its ten separations are not the documented defect, and of six negatives adjudicated by hand three were overturned by an input written after reading the patch: "not separated" mostly means "not reached". The honest deliverable is the verified cases and a characterised instrument, and `eval/realdefects/README.md` names each remaining fault with its measured exposure.

None of this is admissible evidence and none of it is part of the gate; it is research code that fetches from the network and executes third-party source. The deployment-facing surface remains defined and empty: misbind, silent-fail, bleed and time-to-stage (layer I); refutation, discord, miss-observed, replay-divergence and escape-replay rates (layers R and C); all with their selection biases stated beside them. A deployment still owes the two empirical studies the theorems cannot supply: coupling of its defect models to its generators' real faults, and the escape corpus's coverage of what its finders actually find.

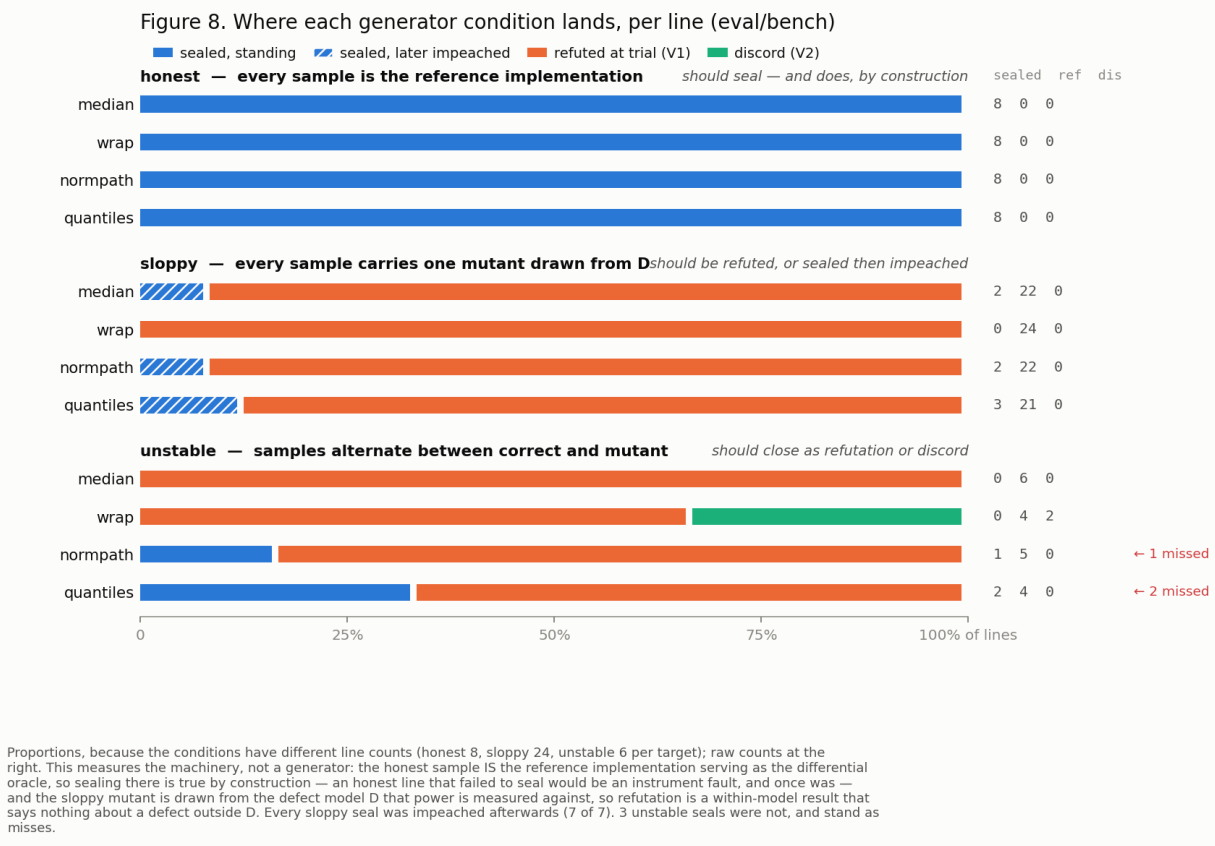
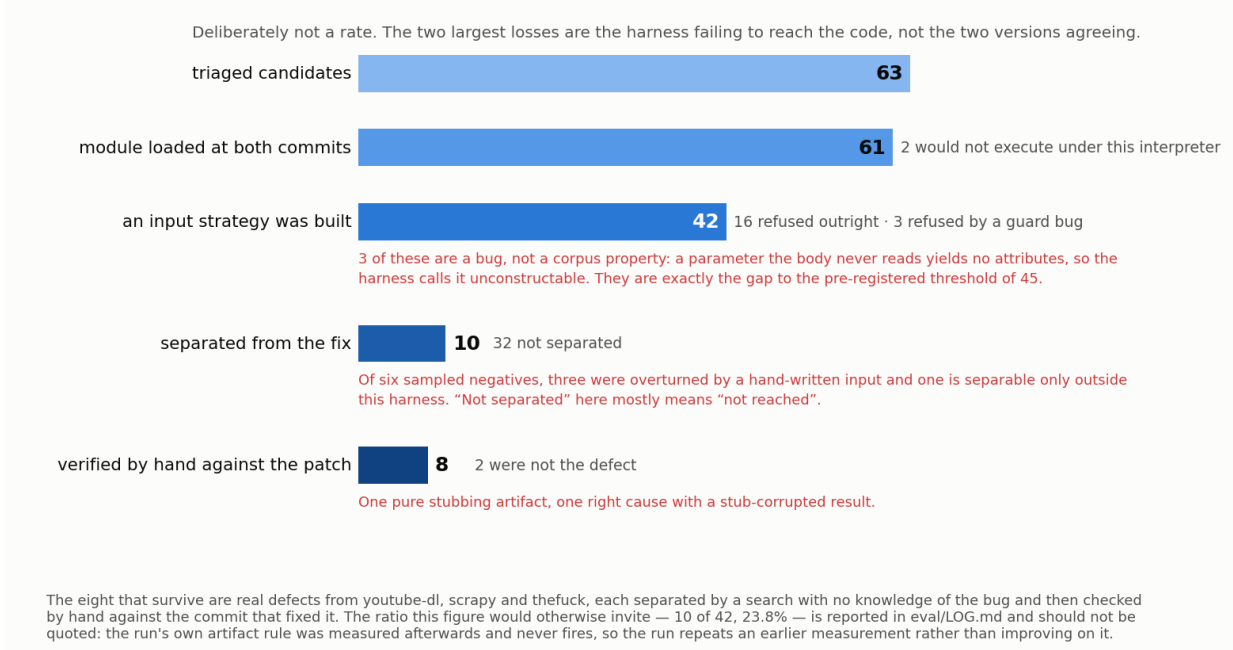


Figure 9. Where 63 real defects went (eval/realdefects)



10. Related work

Consolidated from the layer papers, which carry the full discussions. Foundations: Clark & Wilson (1987) — this stack is their model completed for stochastic workers: layer I is the enforcement rules, layer R the integrity verification procedure with its certification record, layer C the certification-maintenance discipline; Saltzer & Schroeder (1975) fail-safe defaults; Anderson's reference-monitor triad (1972) — complete mediation, tamper-evidence, verifiability — is what §6–§7 instantiate for work-products. Scrutiny: mutation analysis (DeMillo, Lipton & Sayward 1978; Jia & Harman 2011; Just et al. 2014), property testing and randomized checkers (Goldreich–Goldwasser–Ron 1998; Freivalds 1977), proof-carrying code as the power-1 corner (Necula & Lee 1996), semantic entropy as the graded statistic of which concordance is the zero-entropy gate (Kuhn, Gal & Farquhar 2023; Farquhar et al. 2024), N-version limits (Knight & Leveson 1986). Standing: eliminative argumentation and Assurance 2.0 defeater discipline (Goodenough, Weinstock & Klein 2015; Bloomfield & Rushby 2024) — mechanized here, with kernel-refused rather than assessor-flagged forgetting; regression-from-escapes and mined-fault mutation (Tufano et al. 2018; Beller et al. 2021); revocation infrastructure (CRL/OCSP/Sigstore) minus discretion; flaky-test quarantine and SRE error budgets as demotion's fail-open ancestors; adaptive conformal inference (Gibbs & Candès 2021) as the statistical-calibration theorem this stack deliberately does not claim. Contemporary neighbours (2026) circle the gap without the enforcement object — adversarial stage-gated review with LLM refuters, self-consistency admission as a soft score, attestation-aware promotion gates — surveyed in ../RGA/DRAFT.md §10. Evidence-law framing: *Daubert v. Merrell Dow* (1993); Popper (1959). To our knowledge, non-discretionary revocation entailed by a replayable counterexample, and mechanical charge totality against vouching checkers, have no prior formalization.

11. Limitations, and what is not proved

Stated before anything here is read as more than it is. **Truth:** survival is not correctness; impeachment's absence is not soundness; nothing in I, R or C says an artifact is good. **Coupling:** power is exact on a named D and transfers to a generator's real defects only under an empirical hypothesis, stated, not proved — and now attacked directly rather than left standing. §9's real-defect study is that attack, and its result is that the hypothesis remains open: the differential separates eight real historical defects from their fixes, which is evidence that the mechanism reaches real faults at all, but no rate survived scrutiny, five runs each hid an instrument fault behind a plausible-looking figure, and the corpus is filtered to module-level functions (63 of 501), so nothing about a population follows. Coupling is unproved as before; what has changed is that the gap has been attacked with real defects rather than left as an assertion, and that the instrument which would close it is characterised in `eval/realdefects/README.md`, including the faults that stop it. **Representativeness:** the escape corpus is a selected, adversarially steerable subsample of real defects; every aggregate built on it can be steered by parties who file only true things; silence is unreadable except through journaled audits whose reference is the corpus itself. **Reports:** harness honesty, runtime hermeticity, author identity, nonce unpredictability, byte availability and operator governance are assumptions (A10, B1, B2, B7, B9, B11–B13) — with the one asymmetry this stack buys: lies about deterministic components are falsifiable by replay after the fact. **Exposure:** a generator that has memorized a public refuter can game it; the post-artifact seed and package exclusions narrow this and do not close it. **Consumer redirection:** the guarantees stop at the kernels' queries; a deployment whose DAG edges and promotion predicates still read the layer-I store has layer-I guarantees only. The reference server now routes promotion and per-line state through `admissible()`, with unsealable lines labelled layer-I and their reason stated; consumers outside this repository remain an obligation. The server's admission and calibration state is process-lifetime only — the kernels rebuild from their journals (from `_events`, re-guarded), but the reference server persists none of them, so a restart empties the escape ledger and everything C promises about forgetting holds only within one process. **Record completeness:** replay proves the events present form a history the live machine would have accepted, not which history actually ran. A coherent rewrite of root inputs and deletion — of a journal's tail, or of any coherent group of events that no surviving event recomputes against — are indistinguishable from another honest history to replay alone; deletion can raise standing: a dropped escape un-impeaches its line, a dropped refusal un-taints every seal that relied on it. Refusal groups are now in that class unconditionally: closing the un-revocation defect of §7 removed the accidental anchoring that later stamps and installs gave them, so a refusal group's only effect is the taint it places, and deleting it removes exactly that. The residue is one event class wider than in 0.8.1 and the trade is deliberate. A calibration stamp appended for the most recent unmediated seal is accepted for the same reason: nothing earlier contradicts it. v0.5's authenticated publication path is the external anchor: immutable authority journals, three registry-current monotone heads, and a signed composed receipt close these deletions only when callers issue and verify that receipt (`tests/test_authenticated_receipt.py` pins positive, stale, truncation, cross-wiring, atomicity, and wrong-key cases). Replay without that path retains the residue; signer/key and registry compromise or rollback remain assumptions. **Liveness:** nothing here makes any gate reachable. The complete lists are the three "What is not proved" sections, each written before its kernel and extended by its reviews.

12. References

Anderson, J. P. Computer security technology planning study. ESD-TR-73-51, 1972.

Beller, M. et al. What it would take to use mutation testing in industry — a study at Facebook. ICSE-SEIP, 2021.

Bloomfield, R., and Rushby, J. Defeaters and eliminative argumentation in Assurance 2.0. arXiv:2405.15800, 2024.

Clark, D. D., and Wilson, D. R. A comparison of commercial and military computer security policies. IEEE S&P, 1987.

Daubert v. Merrell Dow Pharmaceuticals, 509 U.S. 579, 1993.

DeMillo, R. A., Lipton, R. J., and Sayward, F. G. Hints on test data selection. IEEE Computer, 1978.

Farquhar, S. et al. Detecting hallucinations in large language models using semantic entropy. Nature, 2024.

Freivalds, R. Probabilistic machines can use less running time. IFIP, 1977.

Gibbs, I., and Candès, E. Adaptive conformal inference under distribution shift. NeurIPS, 2021.

Goldreich, O., Goldwasser, S., and Ron, D. Property testing and its connection to learning and approximation. JACM, 1998.

Goodenough, J., Weinstock, C., and Klein, A. Eliminative argumentation: a basis for arguing confidence in system properties. SEI, 2015.

Jia, Y., and Harman, M. An analysis and survey of the development of mutation testing. IEEE TSE, 2011.

Just, R. et al. Are mutants a valid substitute for real faults in software testing? FSE, 2014.

Knight, J. C., and Leveson, N. G. An experimental evaluation of the assumption of independence in multiversion programming. IEEE TSE, 1986.

Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty. ICLR, 2023.

Necula, G. C., and Lee, P. Safe kernel extensions without run-time checking. OSDI, 1996.

Popper, K. The Logic of Scientific Discovery. 1959.

Saltzer, J. H., and Schroeder, M. D. The protection of information in computer systems. Proceedings of the IEEE, 1975.

Tufano, M. et al. Learning how to mutate source code from bug-fixes. arXiv:1812.10772, 2018.

Full per-layer bibliographies: ../DRAFT.md §11, ../RGA/DRAFT.md §12.